Technical Design Document: Data Access and Security Hub (DASH) for ISAC-Enabled Cell Towers

Architecture, Cryptographic Core, Zero-Trust Controls & Interface Specs

1. Introduction & System Overview

1.1 Background

Integrated Sensing and Communication (ISAC) is a foundational technology for 5G-Advanced and 6G networks. By utilizing the same spectrum and hardware infrastructure for both mobile communication and radar-like radio sensing, ISAC-enabled cell towers can detect the velocity, distance, and angle of physical objects. This capability enables applications like drone tracking, traffic monitoring, and intrusion detection.

However, blending communication traffic with environmental sensing data introduces unprecedented security risks and architectural complexities. The **Data Access and Security Hub (DASH)** is the centralized, edge-deployed software framework responsible for governing ingestion, transformation, access control, and cryptographic protection of ISAC data streams directly at the gNodeB/eNodeB edge.

1.2 System Objectives

- **Zero-Trust Data Isolation:** Ensure complete separation between public communication payloads, internal network telemetry, and third-party sensing data.
- **Low-Latency Access Control:** Enforce sub-millisecond access authorization at the edge to support real-time sensing use cases.
- **Privacy by Design:** Anonymize and obfuscate spatial and temporal radar tracking data to protect civilian privacy.
- **Multi-Tenant Delivery:** Safely expose processed sensing APIs to distinct external entities (e.g., municipal traffic management, logistics, defense).

2. System Architecture

DASH operates as a containerized microservices architecture deployed within the Open Radio Access Network (O-RAN) Centralized Unit (O-CU) or Distributed Unit (O-DU) edge compute infrastructure.

2.1 Functional Modules

2.1.1 Sensing Ingestion Engine (SIE)

- **Purpose:** Directly interfaces with the Radio Link Control (RLC) and Medium Access Control (MAC) layers to capture raw In-phase/Quadrature (I/Q) data matrix reflections or pre-processed Radar Point Clouds.

• **Mechanism:** Utilizes Data Plane Development Kit (DPDK) and SR-IOV to bypass kernel overhead, guaranteeing high-throughput ingestion from the RF front-end.

2.1.2 Cryptographic Core (CC)

• **Purpose:** Handles line-rate encryption of sensing streams and establishes hardware-rooted trust.

• **Mechanism:** Integrates with local Hardware Security Modules (HSM) or Trusted Platform Modules (TPM 2.0). Implements AES-256-GCM for data-at-rest encryption and ChaCha20-Poly1305 for streaming data-in-transit.

2.1.3 Access Control Engine (ACE)

• **Purpose:** Evaluates dynamic data authorization policies using Attribute-Based Access Control (ABAC).

• **Mechanism:** Runs an embedded Open Policy Agent (OPA) compiler to cross-reference tenant requests against real-time variables like time of day, geographic coordinates (Geofencing), and threat levels.

2.1.4 Secure API Gateway

• **Purpose:** Exposes authorized sensing streams to authenticated core network applications and external consumers.

• **Mechanism:** Envoy-based proxy enforcing strict mutual TLS (mTLS), token-based authentication, and token rate-limiting.

3. Technical Data Flows

1. Ingestion Step: The cellular transceiver detects a reflection matrix from a physical target. The MAC layer encapsulates this into an internal ISAC-frame format and forwards it to the SIE via eBPF kernel probes.

2. Normalization & Anonymization: The SIE aggregates frames, constructs a sparse 4D radar tensor (X, Y, Z, Doppler Velocity), and strips out any accompanying MAC or cellular UE identifier fields to preserve communication privacy.

3. Cryptographic Tagging: The CC applies an ephemeral session-key wrapper to the data block and computes an HMAC signature for data-integrity validation.

4. Policy Enforcement & Delivery: The API Gateway intercepts the pull/push request from a consumer, evaluates the client certificate, validates the OPA rules via ACE, conditionally downsamples or masks specific geographic sectors (e.g., hiding military installations or private properties), and streams the payload.

4. Security Architecture & Zero-Trust Framework

4.1 Authentication & Authorization

Identity Management: Every client component, consumer application, and adjacent gNodeB node must authenticate via mTLS with X.509 certificates managed by a local SPIRE/Spiffe production identity control plane.

Authorization Scheme: Abandoning classic RBAC, DASH enforces an Attribute-Based Access Control (ABAC) profile defined in JSON/Rego, mapping subject attributes (organization, clearance), resource attributes (resolution, geographic sector), and environmental attributes (current UTC hour, cell tower load).

4.2 Data Protection and Privacy Realization

Spatial Geofencing & Blanking: The ACE maintains dynamic polygon coordinates. If a target tracking vector crosses into an unauthorized coordinate space (e.g., a residential sector), the data stream is programmatically zeroed out or blurred to prevent privacy violations.

Homomorphic Encryption for Multi-Tenancy: For highly sensitive workloads where the edge processing node itself is semi-trusted, DASH implements Functional Encryption (FE) variants, enabling third-party analytics algorithms to run pattern matching on raw Doppler spectra without decrypting the underlying geometric shapes.

4.3 Threat Matrix & Mitigations

Threat	Vector	Mitigation Strategy
Sensing Data Spoofing	RF Injection / False target emulation	Physical layer signature verification (Chirp-sequence hopping validation) and cryptographic frame hashing at the RLC layer.
Eavesdropping on Raw Streams	Local edge container breakout	Network isolation using eBPF/Cilium network policies; Hardware-enforced Confidential Computing memory spaces (AMD SEV-SNP / Intel SGX) for DASH runtimes.
Side-Channel Location Leakage	Timing attacks on API requests	Constant-time response padding inside the Secure API Gateway.
Denial of Service (DoS)	Overwhelming sensing compute requests	Priority-based token bucket rate limiting tied directly to the O-RAN Near-RT RIC SLA controller.

5. Technical Stack & Deployment Profile

• **Container Infrastructure:** Cloud-native microservices architecture orchestrated via lightweight K3s (Kubernetes for Edge) or deployed as native xApps within an O-RAN compliant Near-Real-Time Radio Intelligent Controller (RIC).

• **High-Performance Ingestion Plane:** C++20 compiled with native AVX-512 vector extensions for raw signal parsing.

• **Control and API Management Plane:** Go-based microservices utilizing the Gin-gonic framework and Envoy proxy.

• **Storage Architecture:** Hot Tier (Sensing telemetry) utilizes TimescaleDB / InfluxDB for time-series logging of spatial metrics. Transient Queue utilizes Apache Kafka / Redpanda running inside the edge node for ultra-low latency streaming decoupling.

6. Interface and API Specifications

DASH exposes data primarily via low-overhead gRPC protocols over HTTP/2 to optimize throughput over edge networks.

```
syntax = "proto3";
```

```
package dash.isac.security.v1;

service IsacSensingStreamService {
  // Streams real-time, policy-filtered tracking targets
  rpc StreamDetectedTargets(TargetSubscriptionRequest)
      returns (stream TargetTelemetryResponse);
}

message TargetSubscriptionRequest {
  string client_tenant_id = 1;
  GeoPolygon requested_bounding_box = 2;
  float minimum_velocity = 3;
}

message GeoPolygon {
  repeated Coordinate coordinates = 1;
}

message Coordinate {
  double latitude = 1;
  double longitude = 2;
  double altitude = 3;
}

message TargetTelemetryResponse {
  int64 timestamp_ns = 1;
  string tracking_id = 2;
  Coordinate position = 3;
  repeated float velocity_vector = 4; // [Vx, Vy, Vz]
  float radar_cross_section = 5;
  bytes cryptographic_signature = 6;
}
```

7. Compliance & Auditability

DASH records an immutable ledger of all access decisions, system status changes, and administrative actions.

Every policy transaction executed by the Access Control Engine is serialized into a condensed binary JSON format and appended to a **write-once-read-many (WORM)** audit log. These audit trails are cryptographically chained together using a Merkle tree topology, allowing central network security operation centers (SOC) to detect any log tampering or unauthorized access attempts within the cell tower's localized edge compute engine.